

Security Analysis

MCP for Unity — connecting an AI assistant to the Unity Editor

Subject	MCP for Unity bridge (Python server + Unity C# Editor package)
Question	Where are the risks, and how do I connect Claude to Unity safely?
Repository	CoplayDev/unity-mcp (fork: skevin220/unity-mcp)
Scope	Server/ (Python), MCPForUnity/Editor (C#), CustomTools/
Method	Static source review of transport, auth, command dispatch & tool surface
Date	2026-06-19

Bottom line. MCP for Unity is *safe-by-default at the network edge* (loopback-only binds, HTTPS-required and API-key auth for remote-hosted mode, solid path-traversal defenses). The real risk is **by design**: once any client reaches the local bridge it gets **unauthenticated, unconfirmed, arbitrary code execution inside the Unity Editor process**. There is no per-command auth and no approval prompt on the local path. Your security boundary is therefore “any process that can open a loopback socket on your machine.” Connect Claude safely by keeping binds on loopback, running on a single-user trusted machine, leaving code-execution tools disabled, and letting your MCP client’s per-tool approval UI be the human gate.

1. How the connection works (trust model)

MCP for Unity is two codebases that form one pipe between an AI assistant and the live Unity Editor:

- **AI assistant (Claude)** → speaks MCP over **stdio** or **HTTP** to ...
- **Python MCP server** (`Server/src/`) — exposes the tools/resources, then forwards each command to ...
- **Unity C# Editor package** (`MCPForUnity/`) over a **loopback TCP bridge** (stdio mode, port 6400) or an outbound **WebSocket** to a shared hub (HTTP mode), which finally calls ...
- **Unity Editor API** — scenes, assets, scripts, compilation, play mode.

Two transport modes exist, with very different exposure:

Mode	Binds / reaches	Auth	Default
Stdio (legacy)	Loopback TCP <code>127.0.0.1:6400</code> ; one Python process per client	None	Yes
HTTP local	Loopback <code>127.0.0.1:8080</code> ; shared server, WS hub at <code>/hub/plugin</code>	None	Opt-in
HTTP remote-hosted	Operator-chosen host; HTTPS required by default	API key (X-API-Key), fail-closed	Opt-in

Key consequence: the local transports (stdio TCP and HTTP-local) carry no authentication and no authorization. The Python server is essentially a pass-through; whatever command type it receives is forwarded to Unity, and the C# side executes any registered, enabled tool by name. The only enforced gate on the local path is a per-tool enable/disable toggle — not an identity or approval check.

2. Risk register (ranked)

Each finding below is something a malicious or buggy actor with access to the local bridge — or a prompt-injected/confused AI — could exploit. File references point to the reviewed source.

CRITICAL

HIGH

MEDIUM

LOW

CRITICAL

R1 · runtime_compilation custom tool = unfiltered Editor RCE, enabled by default

Compiles arbitrary C#, `Assembly.Load(bytes)`, adds components and invokes static methods. It is declared `Group="core"` and is a built-in type, so it is **enabled by default** whenever the Roslyn DLLs are present — with **no safety checks at all**. `execute_with_roslyn` runs whenever Roslyn is installed. This is the single most exposed code-exec path.

Location: `CustomTools/RoslynRuntimeCompilation/ManageRuntimeCompilation.cs`

HIGH

R2 · execute_code: arbitrary C# with a bypassable blocklist

Compiles and runs arbitrary C# in-process. “Safety” is naive case-insensitive string matching of ~11 patterns (`File.Delete`, `Process.Start`, `while(true)`...) and the caller can set `safety_checks=false` to disable it. Trivially evaded (reflection, string-splitting). Lives in the `scripting_ext` group (off by default on the server), but a client talking straight to the Unity socket can invoke it if enabled.

Location: `MCPForUnity/Editor/Tools/ExecuteCode.cs` : 41-55, 94 · `Server/.../tools/execute_code.py`

HIGH

R3 · manage_script compile-on-write → auto-execution

Core, default-enabled. Writes arbitrary C# under `Assets/` and triggers recompile. An attacker can plant an `[InitializeOnLoad]` / static-constructor class that Unity **auto-compiles and auto-runs on the next domain reload** — code execution without ever calling `execute_code`. Only syntax/best-practice validation, no security filtering.

Location: `MCPForUnity/Editor/Tools/ManageScript.cs` : 393, 794 · `Server/.../tools/manage_script.py`

HIGH

R4 · No authentication on the local stdio TCP bridge; connections hijack

Binds `127.0.0.1:6400` (loopback — good), but any local process that connects and sends a framed JSON command gets it executed. There is no token or origin check, and a new connection actively kills the previous one, so any local process can hijack the bridge and drive the full toolset.

Location: `MCPForUnity/Editor/Services/Transport/Transports/StdioBridgeHost.cs` : 373, 549-564

HIGH

R5 · Unauthenticated /api/command channel in HTTP-local mode

A REST route that forwards an arbitrary command type + params to the connected Unity session — including `execute_code` / `execute_menu_item`. It has zero auth and relies entirely on the loopback bind for safety. The instant the server is bound to a non-loopback interface this becomes network-reachable RCE.

Location: `Server/src/main.py` : 408-526 (mounted only when NOT remote-hosted)

MEDIUM

R6 · execute_menu_item: blacklist of one

Runs almost any Editor menu command via `EditorApplication.ExecuteMenuItem`. The deny-list contains a single entry (`File/Quit`); builds, asset-deletion menus, package operations and third-party plugin menu items are all allowed.

Location: `MCPForUnity/Editor/Tools/ExecuteMenuItem.cs` : 16-38

MEDIUM**R7 · No confirmation/approval gating on the MCP path**

A confirm-prompt helper exists — but only for the developer CLI. The MCP tools that Claude actually calls have **no** server-side approval step. Destructive actions are flagged with `destructiveHint=True` annotations, but honoring them is entirely up to the MCP client UI.

Location: `Server/src/cli/utils/confirmation.py` (CLI only) · no equivalent for MCP tools

MEDIUM**R8 · Silent exposure if bound off-loopback**

Host is operator-controllable via `--http-host / UNITY_MCP_HTTP_HOST`, and the shipped `docker-compose.yml` binds `0.0.0.0:8080`. No warning is logged when a non-loopback bind is chosen — a single mis-set flag turns the unauthenticated control plane (R5) into remote RCE.

Location: `Server/src/main.py` : 849-851, 922-926 · `docker-compose.yml` : 13

MEDIUM**R9 · Group visibility is not a security boundary on the Unity side**

The “core-only by default” grouping is enforced where the Python server advertises tools, not in Unity. The Unity dispatcher checks only the enable/disable flag, and built-in tools default to enabled. A client speaking directly to the socket can call non-core tools (e.g. `execute_code`) regardless of group.

Location: `MCPForUnity/.../TransportCommandDispatcher.cs` : 354-362 · `ToolDiscoveryService.cs` : 251

LOW**R10 · Telemetry on by default may leak paths**

Anonymous, opt-out telemetry to `api-prod.coplay.dev` is enabled by default. No source or full project paths are sent, but truncated Unity *error strings* (200–500 chars) can embed file/asset paths. Disable with `DISABLE_TELEMETRY=true`.

Location: `Server/src/core/telemetry.py` · `config.py` : 61-63

LOW**R11 · Supply-chain: unpinned-by-hash fetch-and-run**

The server is launched via `uvx` (fetches+runs the package from PyPI). Roslyn DLLs are downloaded from `nuget.org` over HTTPS with pinned versions but **no hash/signature verification**, then written into `Assets/Plugins/Roslyn` and loaded. Compromise of PyPI/NuGet or the channel would run as you.

Location: `MCPForUnity/Editor/Setup/RoslynInstaller.cs` : 88-119 · `ServerCommandBuilder.cs`

LOW**R12 · No session isolation in local multi-client mode**

By design for single-user local use: with no `user_id`, all sessions share one map and any client can be routed to any connected Unity instance. Remote-hosted mode is correctly user-scoped and fails closed. Only a concern if a local server is shared across trust domains.

Location: `Server/src/transport/plugin_registry.py` : 173-180

3. What is already safe-by-default (the good)

The project is deliberately fail-closed at the network edge. These protections are real and worth keeping:

- **Loopback-only binds by default** for both stdio (127.0.0.1:6400) and HTTP-local. LAN bind (0.0.0.0) is an explicit opt-in (“Allow LAN Bind”).
- **Remote-hosted mode requires API keys** (X-API-Key), validated against an external endpoint, **fails closed** on any error, caches only definitive results, and redacts keys in logs.
- **HTTPS required for remote endpoints** by default; plaintext http:// needs an explicit “Allow Insecure Remote HTTP” opt-in.
- **Path traversal is well defended** in `manage_script` / `manage_asset`: paths are canonicalized then prefix-checked against the Assets root, with symlink/reparse-point guards. You cannot write `.cs` outside the project, and asset ops are `AssetDatabase`-scoped.
- **No dangerous deserialization** on the wire — JSON only, no pickle/eval/exec on network input. Stdio framing is length-bounded (64 MiB cap).
- **Telemetry is anonymous and opt-out** (random UUID, no account/IP in payload, bounded & fire-and-forget).
- **No shell-out from tool handlers** — the only subprocess use is list-form argv in an editor-focus helper, not reachable from AI input.

4. How to connect Claude to Unity safely

A practical checklist, roughly in priority order. The goal is to keep the powerful-by-design Editor access behind a trusted boundary and a human approval step.

- 1 Keep every bind on loopback.** Never set `--http-host 0.0.0.0` / `UNITY_MCP_HTTP_HOST` to a routable address, and leave “Allow LAN Bind” off. Do not use the shipped `docker-compose.yml` (0.0.0.0) on any reachable network without putting auth in front.
- 2 Treat the whole machine as the trust boundary.** Because the local bridge has no auth, anyone or any process with local access gets Editor-level code execution. Run Claude+Unity only on a single-user machine you trust; avoid shared/CI/multi-user hosts.
- 3 Leave code-execution tools disabled unless you need them.** In the MCP Tools window keep the `scripting_ext` group off (this is `execute_code`). If you don’t need them, also disable `execute_menu_item` and `play-mode`. Remember `manage_script` alone can still execute code via `[InitializeOnLoad]`.
- 4 Do NOT install the Roslyn runtime_compilation custom tool unless essential.** It is the highest-exposure path (R1): unfiltered RCE, core group, enabled the moment its DLLs exist. If you don’t need runtime compilation, don’t add `CustomTools/RoslynRuntimeCompilation` or the Roslyn DLLs.
- 5 Use a client that surfaces approvals — and require them.** The server provides no gating; your MCP client’s per-tool approval UI is the only interactive human gate. In Claude Code/Desktop, review each destructive tool call rather than auto-approving. Honor the `destructiveHint` annotations.
- 6 Work in a dedicated, version-controlled project.** The AI can create/delete scripts and assets, run menu items and enter play mode with no confirmation. Point it at a throwaway/feature project under git with backups — not your production project — so any mistake is recoverable.
- 7 Lock down remote-hosted mode if you use it.** Keep API-key auth on (default), keep HTTPS, never enable “Allow Insecure Remote HTTP”, and make sure the validation endpoint returns trustworthy `user_ids` (that is what enforces per-user isolation).

- 8 Disable telemetry if you handle sensitive code.** Set `DISABLE_TELEMETRY=true` (or `UNITY_MCP_DISABLE_TELEMETRY` / `MCP_DISABLE_TELEMETRY`) so truncated error strings containing paths never leave the machine.
- 9 Pin and trust your sources.** Install the package from a source you control (the git URL / pinned tag), and be aware `uvx` pulls the server from PyPI and the Roslyn installer pulls DLLs from NuGet without hash verification. Restrict outbound network if that matters to you.
- 10 Stay current.** Only main and the latest beta are supported. Keep both the UPM package and the Python server updated for security fixes; report issues privately to security@coplay.dev.

Scope & caveats. This is a static source review of the repository at the reviewed branch, focused on the transport, authentication, command-dispatch and tool-execution surfaces. It is not a dynamic pen-test and does not cover third-party dependency CVEs, the hosted `coplay.dev` backend, or the client applications themselves. “Risk” here means capability available to whoever/whatever drives the bridge; many of these behaviors are intentional product features, not bugs.